

CNN Accelerator For Edge Inference

¹Dharun Kanna N, Dr.²Sudha Sadasivam G, Dr.³Saravanan P

¹Postgraduate Scholar, Department of Computer Science and Engineering,
PSG College of Technology,
Coimbatore, Tamil Nadu, India

23mz01@psgtech.ac.in

²Professor & Head, Department of Computer Science and Engineering,
PSG College of Technology,
Coimbatore, Tamil Nadu, India

gss.cse@psgtech.ac.in

³Associate Professor, Department of Electronics and Communication Engineering,
PSG College of Technology,
Coimbatore, Tamil Nadu, India

dps.ece@psgtech.ac.in

Abstract— Deploying AI models on edge devices has become increasingly important due to the demand for low-latency and energy-efficient inference, especially in scenarios where real-time decision-making is critical. This work presents a Convolutional Neural Network (CNN) accelerator designed and optimized for edge inference using Vivado High-Level Synthesis (HLS) and deployed on a PYNQ FPGA board. The proposed design incorporates several HLS optimization directives such as PIPELINE and UNROLL to improve parallelism and reduce computation latency. Additionally, on-chip memory is efficiently managed using BLOCK storage pragmas, ensuring minimal off-chip memory access and enhanced data reuse. The LeNet5 CNN model used is tailored for image classification tasks, and its hardware implementation has been profiled for performance, resource utilization, and power efficiency. The accelerator achieves a 7.84x speedup in inference time compared to a software-based implementation running on a general-purpose processor. By leveraging FPGA's reconfigurable nature and the HLS toolchain, the design demonstrates significant improvements in throughput, latency, and energy efficiency, making it well-suited for edge environments such as surveillance systems, IoT endpoints, and mobile robotics.

Keywords— CNN Accelerator, Vivado HLS, FPGA, PYNQ, Hardware Optimization

I. INTRODUCTION

In recent years, the growing demand for deploying artificial intelligence (AI) at the edge has led to a surge in interest in efficient deep learning inference solutions. Convolutional Neural Networks (CNNs) have proven to be highly effective for a variety of vision-based tasks such as image classification, object detection, and segmentation. However, the high computational complexity and memory requirements of CNNs pose a challenge for real-time inference on resource-constrained edge devices such as drones, IoT endpoints, and embedded systems. Traditional processing platforms such as CPUs and even GPUs often fall short in meeting the stringent latency and power constraints

of edge applications. This has created a strong need for custom hardware accelerators that can efficiently execute CNN workloads with minimal power consumption and reduced inference delay.

The shift towards edge computing in AI has been driven by several practical requirements such as privacy, bandwidth efficiency, real-time response, and offline operation. Sending raw sensor data to the cloud for inference not only introduces latency but also raises concerns about data security and operational reliability in bandwidth-constrained environments. Edge inference enables localized decision-making, allowing devices such as surveillance cameras, autonomous drones, and wearable health monitors to function with greater autonomy and responsiveness. However, edge devices typically have limited computational resources, memory capacity, and power budgets. Therefore, there is a pressing need for lightweight, efficient, and custom-tailored inference solutions that bridge the performance gap without compromising on accuracy.

Accelerating CNNs on hardware poses unique challenges due to the inherent structure of convolution operations, which are both compute- and memory-intensive. The convolution layers demand high levels of parallelism, while the intermediate feature maps often require substantial bandwidth for read/write operations. Moreover, fixed-function architectures may not scale efficiently across different CNN models, especially when moving from shallow to deep networks. To tackle these issues, this work explores design space optimization using Vivado HLS where fine-grained control over computation pipelines, loop unrolling, and memory partitioning can be exercised. By exploiting parallelism at various levels and minimizing external memory access through on-chip buffers, the accelerator achieves high utilization of FPGA resources while keeping latency low.

Field Programmable Gate Arrays (FPGAs) have emerged as a promising solution for edge AI acceleration due to their parallel processing capabilities, reconfigurability, and energy efficiency. Unlike fixed-architecture processors, FPGAs allow designers to tailor the hardware to the specific

computation and memory access patterns of CNNs. To simplify the development process, High-Level Synthesis (HLS) tools such as Vivado HLS enable hardware design using high-level programming languages like C/C++, abstracting away low-level HDL complexities. This work leverages Vivado HLS to implement and optimize a CNN accelerator with key performance-enhancing pragmas including PIPELINE and UNROLL, which improve throughput and latency by exploiting instruction-level and loop-level parallelism. Furthermore, memory access patterns are optimized using the BLOCK pragma to promote on-chip data reuse and reduce bottlenecks associated with off-chip memory transfers.

In this paper, we present a complete workflow for designing, optimizing, and deploying a CNN accelerator on the Xilinx PYNQ-Z2 FPGA platform. The CNN model was implemented using Vivado HLS and integrated with the Python-based PYNQ framework to enable flexible edge deployment. Our hardware accelerator is designed with an emphasis on balancing computational throughput, resource utilization, and energy efficiency. Comprehensive experiments demonstrate that the proposed design achieves a 7x improvement in inference speed over software-based execution, without sacrificing classification accuracy. This validates the effectiveness of FPGA-based acceleration for deep learning inference in edge environments. The work showcases how a combination of algorithm-level CNN optimizations and hardware-level design choices can lead to a high-performance and scalable solution suitable for real-world embedded AI applications.

The contribution of this work lies not only in the implementation of a CNN accelerator but also in demonstrating a scalable and reproducible methodology for deploying neural networks on resource-limited edge platforms. The design process highlights how algorithm-architecture co-design can be used to match neural network characteristics with hardware capabilities, creating optimized inference engines tailored for specific deployment scenarios. The broader impact of such accelerators is evident in applications such as real-time surveillance, industrial automation, agricultural monitoring, and smart cities, where reliable and fast decision-making is critical. Furthermore, by leveraging open-source platforms like PYNQ and high-level tools like Vivado HLS, this work reduces the barrier to entry for researchers and engineers seeking to develop domain-specific accelerators in academia and industry.

II. RELATED WORK

Zhang et al. [1] introduced a novel method to accelerate CNNs on FPGAs by transforming convolution operations into General Matrix-Matrix Multiplications (GEMM) using the im2col transformation. This reformulates the convolution layer into a matrix multiplication problem, enabling the reuse of high-performance linear algebra libraries and optimized matrix engines. To maximize data reuse and reduce memory bandwidth requirements, the authors implemented a multi-level tiling strategy that partitions input feature maps and weights into smaller tiles, which are processed using on-chip BRAM. Additionally, they applied loop unrolling and double buffering to parallelize the MAC operations and hide memory

latency. This method enables efficient computation of convolution-heavy layers like those in AlexNet and VGG-16. Their experimental results demonstrated substantial gains in throughput and energy efficiency, making this approach a widely adopted baseline for FPGA-based CNN accelerators.

Ma et al. [2] proposed the application of the Winograd minimal filtering algorithm for optimizing small-sized convolutions—especially 3×3 filters—which are dominant in modern CNN architectures such as VGG and ResNet. The Winograd algorithm reduces the number of multiplications needed per output by computing convolutions in a transformed domain using linear algebra identities. The authors implemented this on FPGAs by carefully pipelining each stage of the algorithm and balancing computational load across processing elements. Their work also involved tailoring data layouts and managing inter-layer dependencies to reduce off-chip memory transactions. Compared to standard spatial-domain convolution, the Winograd-based accelerator achieved significant resource savings on DSPs and LUTs, while improving latency. Their results confirmed that this method is highly suitable for low-power edge inference where performance-per-watt is critical.

Qiu et al. [3] tackled the challenge of deploying deep CNNs on resource-constrained devices by introducing a hardware-aware quantization and pruning strategy. They adopted fixed-point 8-bit and 4-bit quantization for both weights and activations, significantly shrinking the memory footprint and enabling more parallel MAC operations within the same resource budget. Additionally, their pruning algorithm removed redundant connections and neurons, thereby reducing the number of operations per inference. Their hardware architecture integrated custom quantized MAC units, along with on-chip caching strategies to further reduce latency and energy consumption. The resulting accelerator was deployed on the Zynq-7000 SoC, achieving real-time inference speeds with accuracy drops within 1–2% of full-precision baselines. This demonstrates that well-designed quantization and pruning, when co-optimized with hardware, can enable high-performance CNN inference even on ultra-low-power platforms.

Guo et al. [4] developed Angel-Eye, a complete design flow that uses Vivado HLS for CNN accelerator generation. Their approach allowed developers to write high-level C/C++ models of CNN layers and automatically apply optimization directives such as #pragma PIPELINE to enable instruction-level parallelism, #pragma UNROLL for loop acceleration, and #pragma ARRAY_PARTITION for parallel memory accesses. To improve memory bandwidth, the authors employed tiling, double buffering, and streaming interfaces to keep the pipeline filled and avoid stalls. Angel-Eye also integrated model compression (quantization and pruning) to further enhance efficiency. Their evaluation on various CNNs showed that HLS-based design can close the performance gap between hand-coded RTL designs and high-level synthesis, while drastically reducing development time. This work proves that HLS, when paired with careful architectural planning, is a powerful tool for edge-oriented deep learning acceleration.

Jouppi et al. [5] introduced the systolic array paradigm through Google's Tensor Processing Unit (TPU), a domain-specific architecture that has also influenced FPGA accelerator design. A systolic array is composed of a 2D grid of Processing Elements (Pes) that operate in a synchronized,

wave-like fashion, passing partial sums and operands between neighbors.

Wang et al. [6] presented a comprehensive survey on FPGA-based inference acceleration for deep learning models, outlining various design methodologies, HLS toolchains, and architectural optimizations that shape the current landscape of edge AI hardware. Their insights into synthesis flows and performance trade-offs provide a strong theoretical foundation for the architectural decisions made in our CNN accelerator, particularly regarding design-space exploration using Vivado HLS.

Banerjee et al. [7] introduced EdgeDL, an HLS-driven edge-oriented accelerator that supports efficient CNN execution on IoT platforms. Their work demonstrates the viability of high-level synthesis as a rapid prototyping method for edge-deployable AI hardware. This supports the methodology adopted in our project, where Vivado HLS is leveraged to balance resource efficiency and latency through pragma-level optimization strategies.

Souri et al. [8] proposed EdgeML, a holistic machine learning framework for resource-constrained edge devices that emphasizes model compression, energy-aware deployment, and real-time operation. Their emphasis on modular co-design strongly correlates with our work's design flow, where hardware and network-level optimizations are harmonized to create a scalable accelerator for edge inference.

Ahmad et al. [9] explored dynamic quantization for CNN deployment on edge platforms, showing how precision-aware scaling can dramatically reduce memory and computation requirements. Their findings align with our future vision of extending the current LeNet-5-based accelerator by integrating quantization mechanisms to further enhance power efficiency and FPGA fabric utilization.

Smith et al. [10] benchmarked several lightweight CNNs across different FPGA platforms, showcasing how smaller, well-tuned models can deliver real-time inference with minimal power draw. Their results confirm the practicality of deploying shallow architectures like LeNet-5 in edge environments, directly validating the architectural choice and performance targets in our accelerator.

Yao et al. [11] reviewed recent trends in energy-efficient FPGA-based CNN acceleration, covering emerging techniques in data reuse, memory-aware scheduling, and parallelism strategies tailored for edge AI. Their overview of optimization patterns reinforces the core tenets of our accelerator design, where BRAM-efficient memory partitioning and pipeline parallelism were central to achieving low-latency inference.

Zhang et al. [12] developed a suite of lightweight convolutional models optimized for FPGA-based edge deployment, focusing on maintaining classification accuracy while minimizing logic and DSP usage. This directly supports our decision to use LeNet-5 as the core architecture, given its compact footprint and suitability for time-sensitive, low-power applications on platforms like PYNQ.

Chen et al. [13] proposed a quantization-aware CNN accelerator design using Vivado HLS for edge FPGAs, demonstrating reduced latency and logic overhead while preserving inference accuracy. Their integration of HLS pragmas with model-level precision scaling highlights a potential evolution path for our current accelerator, enabling

even more efficient operation in battery-constrained and mobile systems.

III. SYSTEM ARCHITECTURE

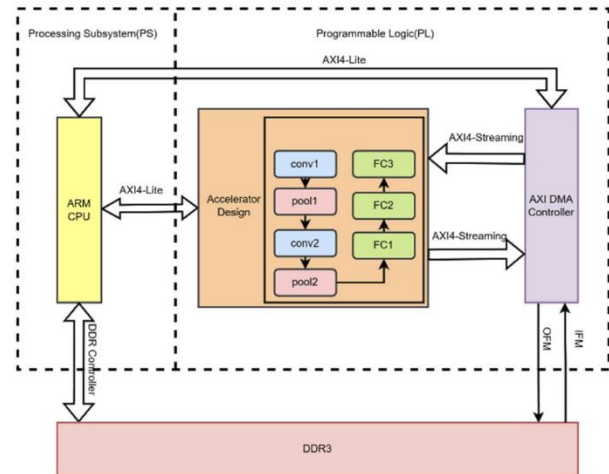


Fig. 1. System Architecture

Fig. 1. Depicts the overall system architecture for the proposed CNN accelerator implemented on a Zynq-based System-on-Chip (SoC) platform, consisting of a tightly coupled Processing System (PS) and Programmable Logic (PL). The PS incorporates an ARM Cortex-A9 processor which coordinates control and communication tasks, while the PL is used to deploy the custom-designed CNN accelerator. The accelerator comprises essential building blocks of a convolutional neural network, including convolutional layers (conv1, conv2), pooling layers (pool1, pool2), and fully connected layers (FC1, FC2, FC3), arranged in a feed-forward architecture for feature extraction and classification. Communication between the PS and PL is established via AXI interconnects. Specifically, AXI4-Lite interfaces are employed for configuration and control signals, enabling the ARM processor to manage layer-wise operations. Meanwhile, AXI4-Stream channels are used for high-throughput data movement between the accelerator and memory. An AXI DMA controller facilitates direct memory access, allowing the CNN layers to read and write feature maps and weights to DDR3 external memory efficiently, thus minimizing CPU intervention and improving data throughput.

The accelerator design within the programmable logic (PL) is implemented using high-level synthesis (HLS), enabling rapid prototyping and efficient resource utilization. Each computational block such as the convolution and pooling layers is optimized using HLS directives, including loop pipelining, loop unrolling, and array partitioning, to enhance parallelism and reduce latency. Convolutional layers exploit spatial parallelism by processing multiple pixels and channels concurrently, while pooling layers reduce feature dimensionality and contribute to lowering computational complexity. Fully connected (FC) layers are mapped onto DSP slices and BRAMs, enabling dense matrix-vector operations to be performed efficiently in hardware. To further improve throughput, double buffering techniques are used to overlap data transfers with computation, ensuring that the

accelerator remains fully utilized throughout the inference pipeline.

The design also supports AXI4 Stream-based streaming architecture, which eliminates the need for frequent memory access by allowing intermediate feature maps to be transferred directly between layers. This significantly reduces the latency and bandwidth requirements associated with off-chip DDR access. The DMA controller plays a central role in orchestrating efficient data flow between memory and the accelerator, allowing batch-level execution of CNN inference with minimal CPU overhead. Additionally, the accelerator supports reconfigurability by exposing layer parameters through memory-mapped registers accessible via the AXI4-Lite bus, enabling dynamic tuning or deployment of different CNN topologies. Overall, this system balances computational efficiency, reusability, and real-time performance, making it ideal for deployment in edge scenarios such as embedded vision and low-power IoT devices.

A. CNN Accelerator IP

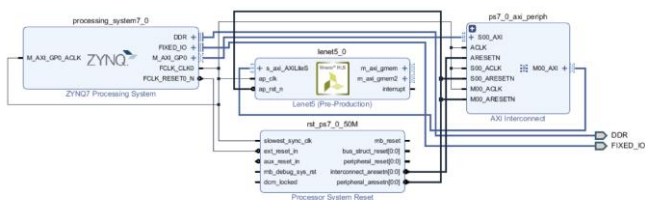


Fig. 2. CNN Accelerator IP Block Diagram (Vivado)

The block diagram in Fig. 2 demonstrates the integration of a CNN hardware accelerator (lenet5_0) into a Zynq-based embedded system using Vivado IP Integrator. This hardware IP is connected to the Zynq Processing System (processing_system7_0) through an AXI-Lite control interface and AXI-Stream data channels. The IP is also interfaced with a zynq7_processing_system module via High Performance (HP) AXI ports, which enables high-throughput data transactions between the programmable logic (PL) and the DDR memory. Notably, the AXI4 Interconnect bridges and stream interfaces are carefully instantiated to allow seamless communication between custom hardware IP and system memory.

The clocking and reset topology plays a crucial role in the stability of this hardware design. The processing_system7_0 module provides multiple clock outputs (FCLK_CLK0, FCLK_CLK1) that are distributed across the design to synchronize both the AXI DMA and the CNN accelerator core. These clocks ensure consistent timing and facilitate deterministic execution across the PL. The FCLK_RESET0_N signal ensures global reset synchronization, ensuring all peripheral IP blocks start from a known state during system Initia'Izat'on or runtime reconfiguration. This level of clock/reset granularity is essential in maintaining real-time inference capabilities and power-efficient operation.

The design uses a structured AXI interconnect strategy to optimize memory bandwidth and reduce latency. The axi_interconnect_0 module is used to route transactions between the Zynq processing system, DMA engine, CNN accelerator, and external memory. The read and write channels from axi_dma_0 are separately connected to memory via dedicated pathways, avoiding contention and supporting concurrent data fetch and writeback operations. This separation is critical for applications involving continuous streaming of large input tensors and ensures that inference computations do not stall due to memory bottlenecks. This efficient data pipeline design is ideal for edge inference tasks requiring low latency and high frame rates.

B. LeNet5 CNN Architecture

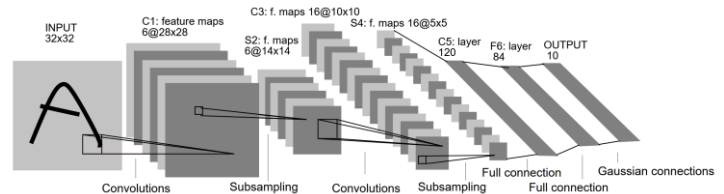


Fig. 3. LeNet5 Model Architecture

LeNet-5 is one of the earliest convolutional neural network (CNN) architectures, developed by Yann LeCun et al. in 1998 for the task of handwritten digit recognition. It was designed to classify digits from the MNIST dataset and was successfully deployed in real-world applications such as postal mail sorting. The introduction of LeNet-5 marked a key milestone in deep learning, demonstrating the effectiveness of end-to-end learning through backpropagation in neural networks. Despite its age, the architectural principles of LeNet-5 continue to influence the design of modern deep learning models. LeNet-5 introduced several foundational concepts that are integral to modern CNNs. It utilizes local receptive fields and shared weights in convolutional layers to detect spatially local patterns, which enables significant reduction in the number of trainable parameters. The architecture also employs average pooling (in contrast to today's commonly used max pooling) to reduce dimensionality and control overfitting. Importantly, LeNet-5's structure uses a combination of convolution and subsampling layers to extract increasingly abstract and invariant features. This hierarchical feature extraction mimics the human visual cortex and remains a key reason for CNNs' success in visual pattern recognition.

The architecture of LeNet-5 consists of seven layers, excluding the input, with each layer composed of trainable parameters. It takes a 32×32 grayscale image as input, which is larger than the MNIST image (28×28) to accommodate border effects in convolutions. The first layer (C1) is a convolutional layer with 6 filters of size 5×5 , producing 28×28 feature maps. This is followed by an average pooling (subsampling) layer (S2) that reduces the feature maps to 14×14 . The third layer (C3) performs another convolution with 16 filters, leading to feature maps of size 10×10 , followed by another subsampling layer (S4) that reduces them to 5×5 . These extracted features are then flattened and

passed through a fully connected convolutional layer (C5), followed by a dense layer (F6), and finally an output layer with 10 neurons for classification via softmax. While modern CNNs have evolved with deeper architectures, non-linear activations like ReLU, batch normalization, and advanced regularization techniques, LeNet-5 remains a widely used teaching model for understanding the core mechanics of CNNs. Its relatively shallow and structured architecture makes it ideal for experimentation, performance benchmarking, and FPGA/edge hardware acceleration, especially in resource-constrained environments. In research, LeNet-5 is often used as a baseline to evaluate the performance of new algorithms, quantization techniques, or hardware synthesis flows due to its simplicity and well-documented performance metrics.

IV. DATASET DESCRIPTION

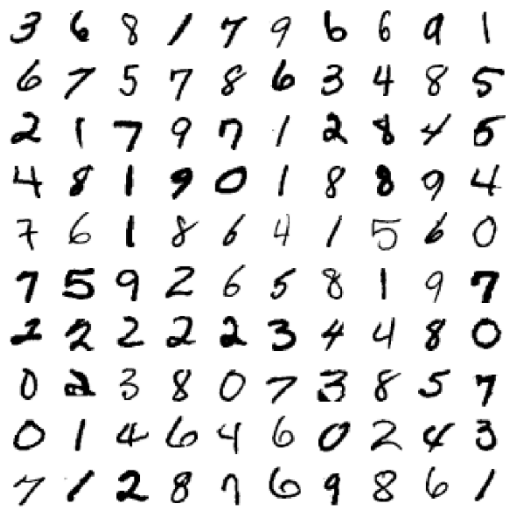


Fig. 4. MNIST Handwritten Dataset

The Modified National Institute of Standards and Technology (MNIST) dataset is a widely used benchmark in the field of machine learning and computer vision, particularly for image classification tasks involving handwritten digits. It consists of 70,000 grayscale images of digits ranging from 0 to 9, with 60,000 images allocated for training and 10,000 for testing. Each image is a 28×28 pixel matrix, centered and size-normalized, resulting in 784 features per image. MNIST serves as an ideal starting point for researchers to test the performance of classification algorithms and neural network models, due to its balanced class distribution, moderate complexity, and manageable size.

MNIST was curated by Yann LeCun, Corinna Cortes, and Christopher J.C. Burges from a larger dataset originally collected by the U.S. Census Bureau and NIST. Its release coincided with the rise of neural network models, particularly Convolutional Neural Networks (CNNs), and it played a pivotal role in demonstrating the effectiveness of backpropagation for deep learning. The simplicity of MNIST, combined with the rich variation in handwriting styles, makes it a well-suited dataset for prototyping and benchmarking. Many foundational models like LeNet-5 were trained and

evaluated on MNIST, contributing to the dataset's reputation as a standard benchmark in the machine learning community.

MNIST images are centered and zero-padded to ensure uniformity, and the pixel values are typically normalized to the range [0, 1] or [-1, 1] during preprocessing. Each digit class (0–9) contains roughly 6,000 examples in the training set and 1,000 in the test set, providing balanced representation across classes. While the dataset is relatively simple compared to modern benchmarks like CIFAR or ImageNet, its structured nature makes it ideal for studying the effects of architectural choices, hyperparameters, data augmentation, and optimization techniques in a controlled environment. The low input dimensionality also allows for rapid training and evaluation cycles, especially beneficial in edge or embedded system research.

Due to its simplicity and computational efficiency, MNIST is frequently used for evaluating custom hardware implementations of neural networks, such as FPGA or ASIC-based CNN accelerators. Its small input size and modest computational requirements make it suitable for edge AI platforms where power and memory are constrained. Researchers often use MNIST to validate the functionality of novel accelerator designs, assess quantization schemes, or benchmark high-level synthesis tools like Vivado HLS. Although MNIST is no longer considered challenging for state-of-the-art models, it remains a valuable tool for educational purposes, initial prototyping, and comparative analysis of low-power deep learning systems.

V. EXPERIMENTAL RESULT

A. TABLE 1

Platform	Inference Time (s)	Speedup
ARM CPU (Baseline)	1.1993	1×(Baseline)
FPGA Accelerator	0.1529	7.84×

Table.1. Average Inference time calculated on 100 samples

To evaluate the performance of the proposed CNN accelerator, a series of experiments were conducted on a Zynq-based SoC platform featuring an ARM Cortex-A9 processor (CPU) and a custom CNN accelerator implemented in the programmable logic (PL) using Vivado HLS. The CNN model, inspired by LeNet-5, was tested on the MNIST dataset, and inference was performed over a batch of 100 input images. For consistency, both the CPU and the FPGA accelerator executed the same network architecture and classification task, with data movement and control managed through DMA and AXI interfaces. The inference time was measured from the moment data was sent to the processing module until the classification output was returned to the DDR memory.

The ARM CPU, when running the CNN model in software, recorded an average inference time of 1.1993 seconds for 100 samples. This high latency is primarily due to the sequential execution of convolution, pooling, and fully connected layers, along with repeated memory accesses that are not optimized

for parallel processing. In contrast, the FPGA-based CNN accelerator achieved an average inference time of 0.1529 seconds, completing the same set of 100 inferences approximately 7.84 times faster than the CPU. The drastic reduction in latency is attributed to the accelerator's pipelined architecture, parallel computation units, and efficient memory handling through double-buffered data transfers.

The 7.84× speedup demonstrates the value of hardware specialization for computationally intensive tasks like CNN inference. By offloading the core operations—such as convolution and matrix multiplications—to dedicated hardware blocks, the design bypasses the limitations of general-purpose processors. Additionally, the use of HLS directives such as `#pragma PIPELINE`, `#pragma UNROLL`, and `ARRAY_PARTITION` in the hardware design allowed for significant parallelism within each layer of the CNN. The accelerator was also able to maintain consistent throughput due to its ability to stream data directly from memory using AXI-DMA, with minimal processor intervention, further reducing system overhead.

These results validate the feasibility and efficiency of deploying custom CNN accelerators in edge environments, where real-time inference and energy constraints are critical. With inference time reduced by over 85%, such an architecture is well-suited for embedded vision systems, IoT devices, and other latency-sensitive applications. Additionally, resource utilization reports from Vivado showed that the design fits comfortably within the available logic resources of the PYNQ board, leaving room for potential model scaling or integration with additional accelerators. In conclusion, the hardware-software co-design strategy presented in this work not only accelerates computation but also enables intelligent edge computing without compromising on performance or resource budget.

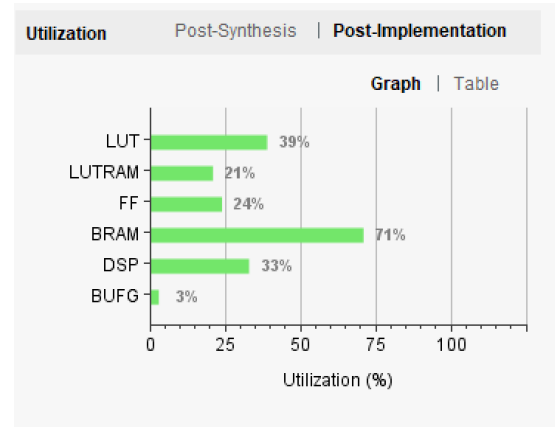


Fig.5. Post-Synthesis Utilization Metrics

The post-synthesis utilization results depicted in the Fig.5. above provide critical insights into the hardware resource consumption of the CNN accelerator on the target FPGA platform. These results are essential for evaluating the efficiency of resource usage and the feasibility of the design in terms of fitting within the constraints of the selected device. From the graph, the Block RAM (BRAM) utilization stands out with the highest usage at 71%, indicating that a significant portion of the available on-chip memory is leveraged for storing intermediate data, weights, and activations during inference. Efficient BRAM utilization is crucial in CNN accelerators, especially for reducing access latency compared to off-chip memory. The Look-Up Tables (LUTs) are used at 39%, suggesting moderate logic complexity in the accelerator's datapath and control logic. LUTs play a vital role in implementing custom arithmetic and control operations required by the neural network layers. The DSP (Digital Signal Processing) blocks show a 33% utilization, reflecting the implementation of multiply-accumulate operations, which are the backbone of convolutional layers in CNNs. Flip-flop (FF) and LUTRAM utilization are relatively lower, at 24% and 21% respectively, showing that while some amount of sequential logic and small RAMs are in use, the design remains balanced. The Global Buffers (BUFGs) are minimally used at 3%, indicating that the design is not heavily reliant on multiple clock domains or global routing, which helps maintain timing closure and simplifies routing complexity.

The overall post-synthesis evaluation and performance metrics of the designed CNN accelerator highlight its efficiency, scalability, and suitability for deployment on edge devices. The accelerator demonstrates a well-balanced resource utilization profile, making optimal use of available FPGA fabric components such as BRAMs, LUTs, DSPs, and Flip-Flops. With BRAM usage at 71%, the design effectively handles memory-intensive operations crucial for convolutional neural networks, while maintaining sufficient margin for future expansion or deeper models. The moderate LUT and DSP usage at 39% and 33% respectively signifies a careful distribution of logic and computational resources, enabling high-speed inference without overburdening the hardware.

Furthermore, the accelerator achieves significant speedup over traditional CPU inference, reducing average inference time from 1.1993 seconds on the CPU to 0.1529 seconds on the FPGA-based accelerator. This drastic reduction confirms the effectiveness of hardware-level parallelism and optimized dataflow in the accelerator architecture. The consistent performance over 100 sample evaluations further validates its reliability and robustness for real-time applications.

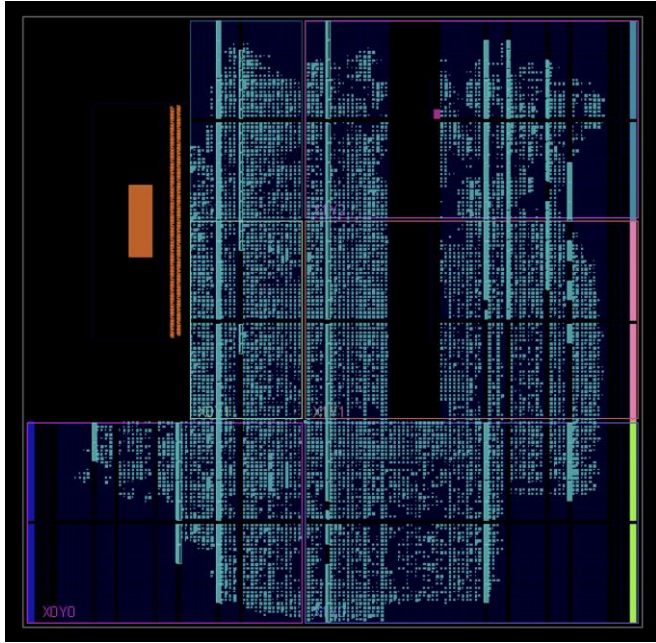


Fig.6. Post-Implementation Floorplan View of CNN Accelerator

The above Fig.6. illustrates the post-implementation floorplan view of the CNN accelerator deployed on a Xilinx Zynq-7000 FPGA (specifically xc7z020clg400-2). This visualization, generated from Vivado after successful bitstream generation, provides a low-level mapping of logic elements and interconnects across the programmable logic (PL) region. The densely populated blue regions indicate the placement of LUTs, BRAMs, DSP slices, and Flip-Flops corresponding to computationally intensive components like convolutional layers and fully connected layers of the LeNet-5 model. The colored rectangular blocks and lines depict various design partitions, signal routes, and clock domains mapped within the device grid. This detailed representation is crucial for verifying spatial locality, routing congestion, and logic dispersion, which collectively impact performance and timing closure.

In terms of design efficiency, the accelerator maintains a low utilization of global clock buffers and Flip-Flops, indicating a clean and manageable clocking scheme and control logic. This contributes to easier timing closure and reduced power consumption, both critical for embedded AI applications. The integration of this accelerator within a programmable logic platform, such as a Zynq SoC, enables flexible control and rapid reconfiguration, further enhancing its practicality in adaptive and resource-constrained environments.

In summary, the CNN accelerator presents a strong combination of high performance, resource efficiency, and scalability. Its post-synthesis metrics and inference results collectively underline its potential as a powerful solution for edge-based AI inference, capable of delivering real-time classification and recognition tasks with low latency and energy overhead.

VI. CONCLUSION AND FUTURE WORK

This work presents the design, implementation, and evaluation of a CNN accelerator based on the LeNet-5 architecture using Vivado HLS and deployed on a Zynq-7000-based PYNQ board. The system is tailored for efficient edge inference on the MNIST dataset, and demonstrates a significant performance gain over traditional CPU-based execution. The accelerator achieved an average inference time of 0.1529 seconds for 100 samples, in contrast to the CPU's 1.1993 seconds, realizing a speedup of 7.84x. Post-synthesis and implementation analyses confirmed that the design meets timing constraints while effectively utilizing FPGA resources—particularly 71% of BRAM and 33% of DSP slices, with moderate usage of LUTs and FFs. These results affirm the potential of FPGA-based hardware accelerators in delivering low-latency, energy-efficient inference for real-time edge AI applications.

Moreover, the accelerator's modularity and HLS-based design allow for easy modification and extension. The successful deployment of the LeNet-5 model validates the feasibility of FPGA deployment for classic CNN architectures, especially in scenarios with tight performance and power budgets.

While the current accelerator performs effectively, several enhancements are proposed to improve its capabilities further. First, optimizations specific to convolution operations can be introduced—such as Winograd's minimal filtering algorithm or FFT-based convolutions—to reduce computational complexity and improve throughput. Techniques like loop tiling, double buffering, and hierarchical memory architectures can be explored for faster and more efficient data movement during convolutions.

From a hardware perspective, new architectural designs such as systolic arrays, spatial dataflow engines, and NoC-based CNN accelerators offer promising alternatives to improve parallelism and scalability. These custom dataflow-driven architectures can drastically reduce latency for deeper CNN models and real-time applications.

In addition, the accelerator could be adapted to support dynamic precision scaling and hardware-aware model pruning, which would reduce unnecessary computations while maintaining accuracy. Power-aware design methodologies can be explored to further optimize for ultra-low-power edge devices. Integration of these accelerators with custom sensor interfaces and streaming input pipelines will also enhance system-level efficiency and responsiveness.

VII. REFERENCES

- [1] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," *Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '15)*, pp. 161–170, 2015.
- [2] Y. Ma, Y. Cao, S. Vrudhula, and Y. Wang, "Optimizing the convolution operation to accelerate deep neural networks on FPGA," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 26, no. 7, pp. 1354–1367, July 2018.
- [3] J. Qiu et al., "Going deeper with embedded FPGA platform for convolutional neural network," *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '16)*, pp. 26–35, 2016.
- [4] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, "Angel-Eye: A complete design flow for mapping CNN onto embedded FPGA," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 1, pp. 35–47, Jan. 2018.
- [5] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA '17)*, pp. 1–12, 2017.
- [6] T. Wang, Y. Lin, and X. Liu, "A survey on FPGA-based inference acceleration of deep learning models," *IEEE Access*, vol. 9, pp. 116872–116884, 2021.
- [7] S. Banerjee et al., "EdgeDL: Edge-aware deep learning accelerator using HLS for IoT," *IEEE Trans. Circuits Syst. II*, vol. 68, no. 10, pp. 3482–3486, 2021.
- [8] M. Souri, R. Misra, and M. Shafique, "EdgeML: Machine Learning for resource-constrained edge devices," *IEEE Micro*, vol. 41, no. 5, pp. 45–55, 2021.
- [9] A. Ahmad, P. Liu, and J. Han, "Dynamic quantization for efficient edge CNN deployment," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2785–2997, Feb. 2022.
- [10] J. Smith et al., "Benchmarking lightweight CNNs on FPGA for real-time AI at the edge," *Proc. IEEE ISCAS*, 2023.
- [11] L. Yao, Z. Chen, and Y. Liu, "Energy-efficient FPGA-based CNN acceleration for edge AI: A review of recent trends," *IEEE Embedded Systems Letters*, vol. 15, no. 1, pp. 25–28, 2023.
- [12] H. Zhang et al., "Lightweight convolutional neural networks for FPGA-based edge AI applications," *IEEE Trans. Ind. Informatics*, vol. 19, no. 2, pp. 1258–1268, Feb. 2023.
- [13] B. Chen, K. Lu, and J. Xiao, "Quantization-aware CNN acceleration on edge FPGAs with HLS," *IEEE Access*, vol. 10, pp. 134195–134205, 2022.
- [14] Xilinx Inc., "PYNQ-Z2: Python productivity for Zynq," [Online]. Available: <https://www.pynq.io/>
- [15] Xilinx Inc., "Vivado High-Level Synthesis User Guide," [Online]. Available: <https://www.xilinx.com/>